

UT.4.1 VISTAS

1. INTRODUCCIÓN	2
2. CREACIÓN DE UNA VISTA	2
3. OPERACIONES CON VISTAS	4
3.1. OPERACIONES DML SOBRE VISTAS	5
4. CONSULTA DE VISTAS EXISTENTES	8

1.INTRODUCCIÓN

Una **vista** es una tabla virtual cuyo contenido es el resultado de una consulta. Al igual que una tabla, una vista consta de un conjunto de columnas y filas de datos.

Una vista no existe como conjunto de valores de datos almacenados en una base de datos, sus filas y columnas de datos proceden de las tablas a las que se hace referencia en la consulta que define la vista y se producen de forma dinámica cada vez que se hace referencia a la vista.

La consulta que define la vista puede provenir de una o de varias tablas, o bien de otras vistas de la base de datos actual o de otras bases de datos.

Ventajas de las vistas:

- **Simplicidad:** suelen usarse para centrar, simplificar y personalizar la percepción de la base de datos para cada usuario.
- **Seguridad:** Las vistas pueden emplearse como mecanismos de seguridad, permitiendo a los usuarios el acceso a algunos datos y manteniendo oculto el resto de la información que no se incluye en la vista.
- **Rendimiento:** el uso de vistas evita codificar repetidamente una consulta almacenando en una vista el resultado de una consulta compleja que incluya información de varias tablas.

Para poder crear vistas tienes que tener asignado el privilegio `CREATE VIEW`, que no está incluido en el rol `RESOURCE`. Por tanto deberás asignarlo a los usuarios que ya tengas creados si no lo has hecho con anterioridad.

2.CREACIÓN DE UNA VISTA

Las vistas se clasifican en:

- **SIMPLES.** Son aquellas que:
 - Sus datos derivan de una sola tabla.
 - No contiene funciones o grupos de datos.
 - Puede realizar sentencias DML a través de la vista.
- **COMPLEJAS.** Son aquellas que:
 - Sus datos derivan de varias tablas.
 - Contiene funciones o grupos de datos.
 - No siempre se pueden realizar sentencias DML a través de la vista.

Una **vista** se define usando un **SELECT**.

La **sintaxis** para crear o modificar una vista es la siguiente:

```
CREATE [OR REPLACE] [FORCE | NOFORCE] VIEW nb_vista
[(columna1 [, columna2]...)]
AS
SUBQUERY
[WITH CHECK OPTION [ CONSTRAINT nb_constraint]]
[WITH READ ONLY];
```

Donde:

- **OR REPLACE:** reemplaza la vista si ya existe. Por tanto la vista puede modificarse sin necesidad de hacer un DROP previo de la misma.
- **FORCE:** crea la vista sin importar si la tabla base existe o no. La vista se crea en estado INVALID.
- **NOFORCE:** crea la vista únicamente si existe la tabla base, es la opción por defecto.
- **WITH CHECK OPTION:** especifica que solamente las filas accesibles a la vista pueden ser insertadas o actualizadas
- **WITH READ ONLY:** Asegura que no se puedan realizar sentencias DML.

Si se omite el nombre de las columnas se adjudican los de las tablas de la consulta. De incluirse, debemos tener en cuenta que entre paréntesis tiene que haber tantos nombres como campos o expresiones devuelva la consulta.

Ejemplo:

```
CREATE OR REPLACE VIEW V_EMPLEADO_DEPARTAMENTO AS
SELECT E.CODEMP, E.NOMEMP, D.CODDEP || ' - ' || D.NOMDEP AS
DEPARTAMENTO
FROM EMPLEADO E
JOIN DEPARTAMENTO D ON E.CODDEP = D.CODDEP;
```

El contenido de una vista se muestra con un SELECT igual que hacemos con el contenido de una tabla:

```
SELECT * FROM NOMBRE_VISTA;
```

Ejemplo:

```
SELECT *
FROM V_EMPLEADO_DEPARTAMENTO;
```

Los nombres para vistas siguen las mismas reglas que cualquier identificador. Para distinguir una tabla de una vista se suele fijar una convención para darle nombre, por ejemplo, colocar el prefijo “V_” o “VISTA_” y luego el nombre de las tablas consultadas en ellas.

Todos los campos y expresiones de la consulta que define una vista **DEBEN** tener un **nombre**, es decir, si tenemos un **campo calculado** debemos darle un **alias** para que ese campo en la vista tenga un nombre.

Si tenemos **varios campos con el mismo nombre**, tendremos que utilizar un **alias** para distinguirlos ya que no podemos tener dos campos con el mismo nombre en la misma tabla.

Las vistas se crean sobre la base de datos activa. Al crear una vista, el gestor de BBDD verifica que existan las tablas a las que se hacen referencia en ella, de no ser así se producirá un error (en realidad como cualquier SELECT).

El ejemplo anterior, en caso de especificar una lista de nombres para las columnas, quedaría así:

```
CREATE OR REPLACE VIEW V_EMPLEADO_DEPARTAMENTO
(CODIGO, NOMBRE_COMPLETO, DEPARTAMENTO) AS
SELECT E.CODEMP, E.NOMEMP, D.CODDEP || ' - ' || D.NOMDEP AS
DEPARTAMENTO
FROM EMPLEADO E
JOIN DEPARTAMENTO D ON E.CODDEP = D.CODDEP;
```

Ejercicio resuelto:

Crear una vista que contenga el nombre del departamento, el salario mínimo por departamento, el máximo por departamento y la media del salario por departamento de las tablas EMPLEADO y DEPARTAMENTO. Redondea los importes a dos decimales:

Solución:

```
CREATE OR REPLACE VIEW V_DEPARTAMENTO_SALARIO
AS
SELECT D.NOMDEP, ROUND(MIN(E.SALEMP), 2) AS SAL_MINIMO,
ROUND(MAX(E.SALEMP), 2) AS SAL_MAX,
ROUND(AVG(E.SALEMP), 2) AS SAL_MEDIO
FROM DEPARTAMENTO D
JOIN EMPLEADO E ON D.CODDEP = E.CODDEP
GROUP BY D.NOMDEP;
```

3. OPERACIONES CON VISTAS

Eliminación de una vista: Al borrar una vista **no se perderán los datos**, porque la vista está basada en tablas subyacentes de la B.D. Sintaxis para eliminar una vista:

```
DROP VIEW NOMBRE_VISTA;
```

Ejemplo:

```
DROP VIEW V_DEPARTAMENTO_SALARIO;
```

Sintaxis para **renombrar** una vista:

```
RENAME ANTIGUO_NOMBRE_VISTA TO NUEVO_NOMBRE_VISTA;
```

3.1. OPERACIONES DML SOBRE VISTAS

Existen las siguientes reglas para realizar sentencias DML sobre las vistas:

- Se pueden realizar sentencias DML sobre las vistas simples.
- No se puede eliminar una fila de una vista, si ésta contiene:
 - Funciones de grupo.
 - Una cláusula GROUP BY.
 - El comando DISTINCT.
- No es posible modificar datos en la vista si la misma contiene:
 - Cualquiera de las condiciones anteriores.
 - Columnas definidas por expresiones.
- No se puede agregar datos si la vista contiene:
 - Cualquiera de las condiciones anteriores.
 - Cualquier columna NOT NULL no incluida en la vista.

Ejemplo: vista simple (modificable)

Creemos una vista para que el jefe de la "Zona Sur" gestione solo a sus empleados.

```
CREATE OR REPLACE VIEW V_PERSONAL_SUR AS
```

```
SELECT
```

```
    CodEmp, NomEmp, SalEmp, CodDep
```

```
FROM EMPLEADO
```

```
WHERE CodDep LIKE '%ZS'; -- Departamentos de Zona Sur
```

```
-- Operación DML válida (UPDATE)
```

```
-- Esto FUNCIONA porque la vista apunta directamente a filas reales  
de la tabla EMPLEADO.
```

```
UPDATE V_PERSONAL_SUR
```

```
SET SalEmp = SalEmp + 100
```

```
WHERE CodEmp = 3;
```

```
-- Operación DML válida (DELETE)
```

```
-- Esto FUNCIONA. Borra al empleado 4 de la tabla base EMPLEADO.
```

```
DELETE FROM V_PERSONAL_SUR
```

```
WHERE CodEmp = 4;
```

Ejemplo: Vista con WITH READ ONLY (protegida)

Aquí forzamos la prohibición explícitamente, aunque la vista fuera técnicamente simple y modificable. Creamos una vista para consultar los centros de trabajo, pero sin permiso para cambiar nada.

```
CREATE OR REPLACE VIEW V_CENTROS_CONSULTA AS
SELECT CodCen, NomCen, PobCen
FROM CENTRO
WITH READ ONLY;
```

```
-- Intento de DML (UPDATE)
-- Esto FALLA con error ORA-42399
UPDATE V_CENTROS_CONSULTA
SET PobCen = 'Madrid'
WHERE CodCen = 'DIGE';
```

Oracle bloquea cualquier intento de INSERT, UPDATE o DELETE sin necesidad de evaluar si la consulta es compleja o no.

Ejemplo: Vista compleja (no modificable por naturaleza)

Una vista estadística que muestra el gasto total en salarios por departamento.

-- Creación (Incluye GROUP BY y Función SUM)

```
CREATE OR REPLACE VIEW V_GASTO_DEPARTAMENTO AS

SELECT

    CodDep,

    COUNT(*) as Num_Empleados,

    SUM(SalEmp) as Total_Salarios

FROM EMPLEADO

GROUP BY CodDep;
```

-- Intento de DML 1: Modificar una columna calculada. Error ORA-01732.

```
UPDATE V_GASTO_DEPARTAMENTO

SET Total_Salarios = 50000
```

```
WHERE CodDep = 'VENZS' ;
```

-- Intento de DML 2: Eliminar una fila. Error ORA-01732.

```
DELETE FROM V_GASTO_DEPARTAMENTO
```

```
WHERE CodDep = 'VENZS' ;
```

Ejemplo de WITH CHECK OPTION: Crear una vista para gestionar solo a los empleados con salario alto (digamos, superior a 5000€). Queremos evitar que alguien inserte un empleado con un salario inferior a través de esta vista.

```
CREATE OR REPLACE VIEW V_EMPLEADOS_VIP AS
SELECT
    CodEmp, NomEmp, SalEmp, CodDep
FROM EMPLEADO
WHERE SalEmp > 5000
WITH CHECK OPTION CONSTRAINT ck_salario_vip;
```

Si intentamos realizar el siguiente INSERT Oracle lanzará el error: ORA-01402: view WITH CHECK OPTION where-clause violation. *¿Por qué?* Porque el salario (1000) no cumple la condición WHERE de la vista (> 5000), por lo que la fila "desaparecería" de la vista nada más insertarse. Oracle lo prohíbe para mantener la coherencia.

Ejemplo de WITH READ ONLY

Crear un directorio telefónico público. Queremos que secretaria pueda consultar los nombres y extensiones, pero que **bajo ningún concepto** puedan borrar empleados o cambiar sus sueldos desde esta vista.

```
CREATE OR REPLACE VIEW V_DIRECTORIO_TELEFONICO AS
SELECT
    NomEmp,
    ExTelEmp,
    d.NomDep
FROM EMPLEADO e
JOIN DEPARTAMENTO d ON e.CodDep = d.CodDep
WITH READ ONLY;
```

Si alguien intenta borrar a un empleado:

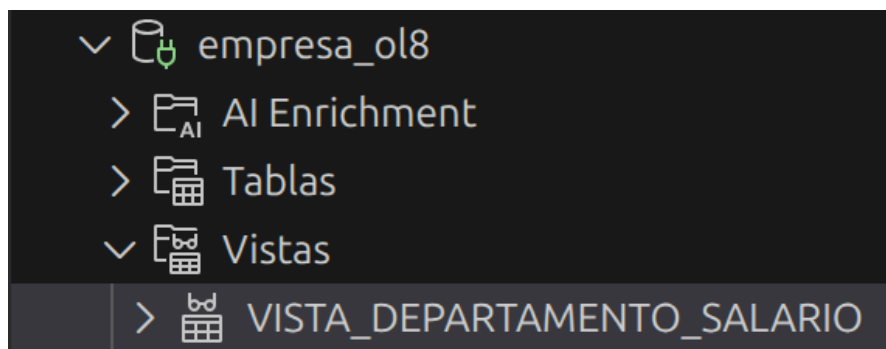
```
DELETE FROM V_DIRECTORIO_TELEFONICO WHERE NomEmp LIKE '%Saladino%';
```

Oracle lanzará inmediatamente: ORA-42399: cannot perform a DML operation on a read-only view.

4. CONSULTA DE VISTAS EXISTENTES

Tienes dos opciones para consultar las **vistas que tienes creadas** o ver el código fuente de una vista antigua.

- Si utilizas un entorno gráfico como Visual Studio Code, SQL Developer o Dbeaver puedes consultar las vistas existentes en la conexión que estás utilizando. Las siguientes imágenes ilustran las vistas existentes y la consulta de la estructura y DDL de una de ellas:



empresa_ol8 > object > EMPRESA > VIEW > VISTA_DEPARTAMENTO_SALARIO

Columnas	Datos	Permisos	Dependencias	Detalles	Disparadores	SQL
Errores						
	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT		
1	NOMDEP	VARCHAR2(40 CHAR)	No	(null)		
2	SAL_MINIMO	NUMBER	Yes	(null)		
3	SAL_MAX	NUMBER	Yes	(null)		
4	SAL_MEDIO	NUMBER	Yes	(null)		

empresa_ol8 > object > EMPRESA > VIEW > VISTA_DEPARTAMENTO_SALARIO

Columnas Datos Permisos Dependencias Detalles Disparadores SQL

Errores

```
1 CREATE OR REPLACE FORCE EDITIONABLE VIEW "EMPRESA"  
2   SELECT D.NOMDEP, ROUND(MIN(E.SALEMP), 2) AS SAL_MINIMO,  
3   ROUND(MAX(E.SALEMP), 2) AS SAL_MAX,  
4   ROUND(AVG(E.SALEMP), 2) AS SAL_MEDIO  
5   FROM DEPARTAMENTO D  
6   JOIN EMPLEADO E ON D.CODDEP = E.CODDEP  
7   GROUP BY D.NOMDEP;
```


- También puedes consultar el diccionario de datos, en particular la tabla USER_VIEWS:

```
SELECT view_name, text  
FROM USER_VIEWS;
```